Ne olvasson el mindent a Claude: gyakorlati útmutató a Projekttérképhez

A Projekttérkép a saját, nyílt forráskódú fejlesztésünk: **kódtérkép a Claude Code-hoz, a graphify motorjára építve.** Feltérképezi a projekted fájljait, függvényeit és kapcsolatait, így a Claude a térképből tudja, hol mi van, és **csak azt a fájlrészletet nyitja meg, ami kell**, nem olvassa be újra és újra az egészet. Repó: github.com/makifej/projektterkep

Mi kell hozzá?

- **Claude Code**, és egy mappa, amiben a projektjeid vannak.
- **uv** (Python-csomagkezelő) a graphify telepítéséhez, és **git** a repó letöltéséhez.
- **Fizetős szolgáltatás nem kell hozzá.** A térkép nem hív nyelvi modellt, nem kerül pénzbe.

1. Telepítsd

```
uv tool install graphifyy
git clone https://github.com/makifej/projektterkep.git
<munkaterület>/projektterkep
```

A szkript a graphify saját Pythonjával fut, ezt érdemes egy változóba tenni:

```
PY=~/.local/share/uv/tools/graphifyy/bin/python
```

2. Készítsd el a térképet

```
$PY projektterkep/projektterkep.py                  # minden projekt
$PY projektterkep/projektterkep.py egyik masik      # csak ezek
$PY projektterkep/projektterkep.py --mappa ~/barmi  # bármelyik mappa
```

- **A munkaterület minden első szintű mappája külön projekt**, mindegyik saját térképet kap, a végén pedig egy összevont gráf készül az összesből.
- **Nincs fájlszám-korlát**, nagy projekten is végigmegy.
- **A változatlan fájlokat nem dolgozza fel újra**, a frissítés pár másodperc.
- **A projektmappákba nem ír**, minden a `graphify-out/` alá kerül.

3. Kérdezz tőle

```
graphify query "hol küldjük el a levelet?" --graph <graph.json>
graphify explain "<elem>" --graph <graph.json>
graphify path "A" "B" --graph <graph.json>
```

A `query` megmondja, hol van valami, az `explain` egy elemről ad részletet, a `path` pedig megmutatja, mi köti össze a kettőt.

4. Tedd be a Claude Code-ba

A repóban lévő `skill/SKILL.md` kész skill. **Másold a `~/.claude/skills/projektterkep/` mappába**, és onnantól a Claude kódos kérdésnél magától a térképből indul: előbb a gráfot kérdezi, aztán célzottan olvas.

5. Mérd meg, mennyit spórolsz

```
$PY projektterkep/projektterkep.py <projekt> --meres "a kérdésed"
$PY projektterkep/projektterkep.py --meres-osszesites
$PY projektterkep/projektterkep.py --elo-meres
$PY projektterkep/kimutatas.py        # http://localhost:4478
```

A mérő ugyanarra a kérdésre két utat vet össze: térkép nélkül és térképpel. Az `--elo-meres` a Claude Code munkamenet-naplóiból méri a valós használatot, a kimutatás pedig mindezt egy helyi oldalon mutatja.

Az első mérésünk (2026-11-07):

Mérés	Térkép nélkül	Térképpel	Spórolás
6 kérdés, 3 projekt	126 271 token	21 268 token	83,2%
Valós használat, 20 lekérdezés	161 721 token	24 569 token	84,8%

Amire figyelned kell

- **A kérdések között nagy a szórás:** volt, ahol 95% fölött spórolt, és volt, ahol a térkép többet adott vissza, mint a sima keresés.
- **A képek, videók és PDF-ek tartalmát nem olvassa ki**, csak fájlként vannak benne a térképben.
- **A térképen a csoportok neve gépies**, mert nem használ nyelvi modellt.
- A token becslés karakter/4, mindkét oldalon ugyanaz a szabály, ezért **a százalék megbízható, az abszolút szám közelítés.**

A Projekttérkép MIT licencű, a graphify a Graphify Labs munkája. Még több AI-eszköz és hír a quicknews.hu-n.